



SURE TRUST VLSI DESIGN INTERNSHIP REPORT

The domain of the Project:
PHYSICAL DESIGN

Team Mentors (and their designation):

Prof. Radhakumari Challa - Founder & Executive director - SURE ProEd

Mr. VEERAMANI R - Project Mentor

Period of the project : April 2026 to June 2026



Declaration

The project titled "**ASIC Physical Design of PicoRV32 RISC-V Processor using OpenLane and SKY130 Technology**" has been carried out under the guidance of the project mentor as part of the **VLSI Physical Design** course organized by **SURE Trust**. The project was undertaken to gain practical knowledge and hands-on experience in ASIC physical design using industry-standard open-source EDA tools and methodologies.

I hereby declare that this report is an original record of the work carried out by me during the course period. The work presented in this report has not been submitted elsewhere for any degree, diploma, or certification. All the information, results, and observations presented are true to the best of my knowledge.

During this project, I successfully completed the assigned laboratory exercises and projects, including Verilog-based digital design, FPGA implementation using Xilinx Vivado, Linux and Git fundamentals, and the complete physical design implementation of the PicoRV32 RISC-V processor using the OpenLane design flow. These activities enhanced my understanding of ASIC design methodologies and strengthened my practical skills in VLSI physical design.

Team Member

Mr. Bhanu Sharma _____ (Signature)

Project Mentor

Mentor Name: _____

Designation: _____

Organization: SURE Trust

Prof. Radhakumari

Executive Director & Founder

SURE Trust

Mentor's Name :

Mr. VEERAMANI R

Designation : Project Mentor



Acknowledgement

I would like to express my heartfelt gratitude to everyone who has contributed through their guidance, encouragement, and constant support towards the successful completion of this project.

I am especially grateful to my mentors and instructors, whose technical guidance has played an important role in shaping my understanding of VLSI and semiconductor design.

I sincerely thank *Dev Chadha Sir* for his valuable guidance in **RTL Design** and for helping me strengthen my understanding of digital design concepts and RTL development.

I am deeply thankful to *Gopinath Polisetty Sir* and *Bhaskar Reddy Sir* for their guidance in **UVM and Design Verification**. Their teachings helped me understand verification methodologies and the importance of systematic validation of digital designs.

I would also like to express my sincere gratitude to *Prudvi Sir* for his guidance in **Analog and Mixed-Signal (AMS) Design**, which broadened my understanding of the interaction between analog and digital domains in semiconductor systems.

I am grateful to *Ashwini Sir* and *Venkatesh Sir* for their valuable guidance in **Design for Testability (DFT) and Synthesis**, which helped me understand the importance of testability and the transformation of RTL into a technology-mapped design.

I extend my sincere thanks to *Prof. Radhakumari Mam* and *Veeramani Sir* for their invaluable guidance in **Physical Design (PD)**. Their support and technical insights were particularly important in developing my understanding of the complete RTL-to-GDSII flow, including floorplanning, placement, clock tree synthesis, routing, timing analysis, and physical verification.

I would also like to express my sincere appreciation to *Satish Sir* and *Anurag Sir* for their **constant support, encouragement, and guidance** throughout my learning journey. Their continuous motivation helped me remain focused and overcome challenges encountered during the project.

Finally, I am deeply grateful to my family, friends, and everyone who supported me directly or indirectly throughout this journey. Their encouragement and faith in me provided the motivation to keep learning and improving.

This project has been a valuable learning experience, giving me not only technical knowledge of **VLSI Physical Design and ASIC implementation**, but also a deeper appreciation for the guidance, collaboration, and perseverance required to grow as an engineer.

I am sincerely grateful to each and every one of you for being a part of this journey.



Table of Contents

Section	Title
1	Introduction
1.1	Background
1.2	Motivation
1.3	Problem Statement
1.4	Objectives
1.5	Scope of the Project
2	Background and Tools
2.1	Introduction to RISC-V
2.2	PicoRV32 Processor
2.3	ASIC Physical Design
2.4	RTL-to-GDSII Flow
2.5	OpenLane
2.6	OpenROAD
2.7	Yosys
2.8	SKY130A PDK
3	Methodology
3.1	Project Overview and Methodology
3.2	Design and Technology Setup
3.3	Design Inputs and Constraints
3.4	RTL-to-GDSII Physical Design Flow
3.4.1	RTL Acquisition and OpenLane Setup
3.4.2	RTL Synthesis and Technology Mapping



Section	Title
3.4.3	Floorplanning and I/O Planning
3.4.4	Power Distribution Network
3.4.5	Placement
3.4.6	Clock Tree Synthesis
3.4.7	Routing
3.4.8	Static Timing and Physical Analysis
4	Results and Discussion
4.1	Synthesis Results
4.2	Floorplanning and Utilization Results
4.3	Power Distribution Network and IR-Drop Results
4.4	Placement Results
4.5	Clock Tree Synthesis Results
4.6	Routing and Wirelength Results
4.7	Post-Route Static Timing Analysis
4.8	Overall Results and Discussion
4.9	Timing Optimization Analysis
5	Reflections and Learning
6	Conclusion and Future Scope
6.1	Conclusion
6.2	Future Scope
7	References



LIST OF ABBREVIATIONS :

Abbreviation	Full Form
ASIC	Application-Specific Integrated Circuit
RTL	Register Transfer Level
PD	Physical Design
PDK	Process Design Kit
CTS	Clock Tree Synthesis
STA	Static Timing Analysis
WNS	Worst Negative Slack
TNS	Total Negative Slack
PDN	Power Distribution Network
IR	Interconnect Resistance / IR Drop
LEF	Library Exchange Format
DEF	Design Exchange Format
SDC	Synopsys Design Constraints
DFT	Design for Testability
UVM	Universal Verification Methodology
AMS	Analog Mixed-Signal
RISC-V	Reduced Instruction Set Computer – Five



Executive Summary

Executive Summary

The **VLSI Physical Design** course organized by **SURE Trust** provided comprehensive training in digital design, FPGA implementation, and ASIC physical design using industry-standard open-source EDA tools. The objective of this course was to bridge the gap between academic knowledge and industry practices by offering hands-on experience in the complete ASIC design flow.

The learning journey began with Linux command-line operations, Git and GitHub version control, and Verilog HDL programming. Fundamental digital circuits, including **4-bit and 32-bit counters**, were designed, simulated, and verified using Verilog. Practical exposure to FPGA design was gained through **Xilinx Vivado**, where a **Birthday Pattern Generator and Finder** was implemented, synthesized, and functionally verified.

The major project focused on the **physical design implementation of the PicoRV32 RISC-V processor** using the **OpenLane** automated ASIC design flow with the **SKY130 Process Design Kit (PDK)**. The project covered every stage of the physical design flow, including synthesis, floorplanning, power distribution network generation, tap cell insertion, placement, clock tree synthesis (CTS), routing, design rule checking (DRC), layout versus schematic (LVS) verification, and static

timing analysis (STA). Various design parameters, such as core utilization and I/O placement, were explored to study their impact on timing, area, and routing performance.

Throughout the course, practical assignments and projects strengthened the understanding of RTL design, FPGA implementation, and ASIC physical design methodologies. The experience also enhanced proficiency with industry-relevant tools, including **OpenLane, OpenROAD, Magic, KLayout, Netgen, Xilinx Vivado, Git, and Linux**.

Overall, this course provided valuable practical exposure to the complete VLSI design workflow, improved analytical and problem-solving skills, and established a strong foundation for pursuing a career in ASIC Physical Design and VLSI engineering.



Introduction

Introduction

The semiconductor industry is continuously advancing towards designing integrated circuits that are faster, smaller, and more power-efficient. While digital logic design defines the functionality of a chip, **ASIC Physical Design** transforms the Register Transfer Level (RTL) description into a manufacturable integrated circuit by determining the placement of standard cells, clock distribution, power planning, routing, and physical verification. Physical design plays a crucial role in achieving timing closure, minimizing power consumption, optimizing area utilization, and ensuring the manufacturability of modern VLSI chips.

The **VLSI Physical Design** course organized by **SURE Trust** was designed to provide practical exposure to the complete ASIC implementation flow using industry-relevant open-source Electronic Design Automation (EDA) tools. The training focused on understanding each stage of the physical design process, enabling participants to gain hands-on experience with the methodologies followed in the semiconductor industry.

The course began with Linux fundamentals, Git version control, and Verilog HDL to build the prerequisite knowledge required for ASIC design. Basic digital design assignments, including the implementation of **4-bit and 32-bit counters**, along with a small FPGA-based **Birthday Pattern Generator and Finder** developed using **Xilinx Vivado**, were completed to strengthen RTL design and verification skills before progressing to physical design.

The primary focus of the course was the **physical implementation of the PicoRV32 RISC-V processor** using the **OpenLane** automated ASIC design flow with the **SKY130 Process Design Kit (PDK)**. The project covered the complete back-end design flow, including synthesis, floorplanning, power distribution network (PDN) generation, tap cell insertion, placement, clock tree synthesis (CTS), global and detailed routing, parasitic extraction, static timing analysis (STA), design rule checking (DRC), and layout versus schematic (LVS) verification. Throughout the implementation, design parameters such as core utilization, aspect ratio, I/O placement, and clock constraints were analyzed to study their impact on timing, area, power, and routing quality.

The project extensively utilized industry-standard open-source EDA tools, including **OpenLane, OpenROAD, Magic, KLayout, Netgen, and Git**, providing practical experience with the ASIC physical design workflow. The hands-on implementation of a RISC-V processor from RTL to GDSII significantly enhanced the understanding of physical design concepts, timing optimization, physical verification, and design closure, thereby establishing a strong foundation for a career in ASIC Physical Design and VLSI engineering.

Problem Statement

Modern semiconductor designs require efficient physical implementation to meet stringent performance, power, and area (PPA) targets. Although RTL design defines the functionality of a digital system, achieving a manufacturable and timing-clean chip requires expertise in physical design methodologies and EDA tools. The objective of this project was to



gain practical experience in implementing a complete ASIC physical design flow and understanding the challenges involved in converting RTL into a fabrication-ready layout.

Scope of the Project

The scope of this project is centered on the complete ASIC physical design implementation of the **PicoRV32 RISC-V processor** using the **OpenLane** design flow and **SKY130 PDK**. The project includes RTL synthesis, floorplanning, PDN generation, tap cell insertion, placement, clock tree synthesis, routing, timing analysis, physical verification, and final GDSII generation. Supporting activities such as Linux, Git, Verilog programming, and FPGA implementation were performed as prerequisite exercises to develop the foundational knowledge required for successful physical design implementation.

Innovation Component

This project demonstrates the complete implementation of an open-source RISC-V processor using a fully open-source ASIC design flow. By integrating OpenLane with the SKY130 PDK, the project provides an end-to-end chip implementation experience comparable to industrial physical design workflows. The ability to perform synthesis, physical implementation, timing optimization, verification, and GDSII generation using open-source tools makes the project highly relevant for learning modern ASIC design methodologies.

Project Objectives

Project Objectives

The primary objective of this project was to gain practical knowledge of the complete ASIC Physical Design flow using industry-standard open-source Electronic Design Automation (EDA) tools. The project aimed to bridge the gap between theoretical concepts and real-world semiconductor design practices by implementing a RISC-V processor from RTL to GDSII.

The specific objectives of the project were:

- To understand the complete VLSI ASIC design flow, from RTL design to physical layout generation.
- To gain hands-on experience with Linux, Git, and GitHub, which are essential tools in modern semiconductor development environments.
- To strengthen digital design fundamentals using Verilog HDL through the implementation and simulation of basic digital circuits.
- To acquire introductory FPGA implementation experience using Xilinx Vivado for RTL verification.
- To study the architecture of the PicoRV32 RISC-V processor before initiating the physical implementation process.
- To perform RTL synthesis using the OpenLane automated ASIC design flow.
- To implement floorplanning by defining core dimensions, aspect ratio, standard cell utilization, and I/O placement.
- To understand power planning through Power Distribution



Network (PDN) generation and tap cell insertion.

- To perform standard cell placement while optimizing congestion and utilization.
- To generate an optimized clock network using Clock Tree Synthesis (CTS) and analyze its impact on timing.
- To perform global and detailed routing for establishing all signal interconnections.
- To verify the physical design using Design Rule Checking (DRC) and Layout Versus Schematic (LVS).
- To perform Static Timing Analysis (STA) and evaluate setup time, hold time, clock skew, and timing closure.
- To analyze the impact of various physical design parameters such as core utilization and I/O placement on area, timing, routing, and overall chip performance.
- To generate the final GDSII layout, representing a fabrication-ready integrated circuit.

PicoRV32 RISC-V processor from RTL synthesis to final GDSII generation.

- Acquired the ability to analyze timing reports, area utilization, routing quality, and physical verification results.
- Strengthened problem-solving skills through practical implementation, debugging, and optimization of the physical design process.
- Built a strong foundation for pursuing a career in ASIC Physical Design and VLSI Engineering.

Expected Outcomes

At the completion of this project, the following outcomes were achieved:

- Developed a clear understanding of the complete ASIC Physical Design workflow.
- Gained practical experience with industry-standard open-source EDA tools, including OpenLane, OpenROAD, Magic, KLayout, and Netgen.
- Successfully implemented the physical design flow of the



Methodology and Results

4.1 Development Environment

Before beginning the ASIC physical design implementation, the required development environment was configured using Linux and various open-source Electronic Design Automation (EDA) tools. The entire physical design workflow was executed on Ubuntu Linux, which provided a stable platform for installing and running the design tools. Version control was managed using Git and GitHub to maintain project files and track modifications throughout the development process.

The OpenLane automated ASIC flow, built on top of OpenROAD, was used to implement the complete physical design of the PicoRV32 RISC-V processor using the SKY130 Process Design Kit (PDK). Additional tools such as Magic, KLayout, and Netgen were utilized for layout visualization, physical verification, and layout-versus-schematic (LVS) checking.

Development Tools

Tool	Purpose
Ubuntu Linux	Development Environment
Git	Version Control
GitHub	Project Repository Management
OpenLane	Automated ASIC Physical Design Flow
OpenROAD	Physical Design Implementation

Tool	Purpose
SKY130 PDK	130 nm Open-source Process Design Kit
Magic	Layout Visualization and DRC
KLayout	GDSII Layout Viewer
Netgen	Layout Versus Schematic (LVS) Verification
Xilinx Vivado	RTL Verification and FPGA Implementation
Verilog HDL	Digital Hardware Description Language

4.2 Course Methodology

The project was carried out in a structured manner, beginning with the fundamentals of digital design and gradually progressing towards complete ASIC physical design implementation.

The initial phase focused on developing familiarity with Linux commands, Git version control, and Verilog HDL. Basic digital design exercises, including the implementation of 4-bit and 32-bit counters, were completed to strengthen RTL design concepts. A small FPGA implementation using Xilinx Vivado was also performed to understand synthesis and hardware verification.

After establishing these fundamentals, the project transitioned to ASIC Physical Design. The PicoRV32 RISC-V processor was selected as the reference design for



implementing the complete OpenLane design flow. Each stage of the physical design process was performed sequentially, including RTL synthesis, floorplanning, PDN generation, tap cell insertion, placement, clock tree synthesis, routing, parasitic extraction, timing analysis, DRC, LVS verification, and GDSII generation.

This methodology provided practical exposure to the complete backend design flow followed in modern semiconductor industries.

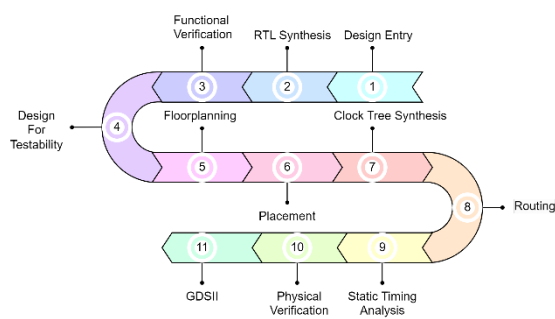


Fig : Overall Methodology

Result

This methodology enabled the successful implementation of the PicoRV32 RISC-V processor through the complete ASIC physical design flow. The project provided

hands-on experience with industrial EDA tools and demonstrated the relationship between design constraints, timing, area, power, and routing quality.

Linux Environment and Basic Commands

The development environment for this project was established using **Ubuntu 24.04 LTS** running on **Oracle VirtualBox**. Linux is the preferred operating system for VLSI and ASIC design because most Electronic Design Automation (EDA) tools are developed and optimized for Unix/Linux environments. Throughout the course, Ubuntu served as the primary platform for installing, configuring, and executing the open-source physical design toolchain.

Before starting the ASIC implementation flow, familiarity with the Linux operating system was developed by learning essential terminal commands, directory management, file handling, user permissions, shell operations, and package management. These skills were fundamental for installing software packages, managing project files, executing design scripts, and debugging the OpenLane physical design flow.

LINUX BASIC COMMANDS: A QUICK REFERENCE

DIRECTORY NAVIGATION (pwd, cd)	FILE LISTING OPTIONS (ls and flags)	CREATING DIRECTORIES (mkdir)				
<pre>bhanu-sharma@bhanu-sharma-VirtualBox\$ cd Openlane bhanu-sharma@bhanu-sharma-VirtualBox\$ pwd /home/bhanu-sharma/Desktop/Openlane bhanu-sharma@bhanu-sharma-VirtualBox\$</pre>	<pre>bhanu-sharma@bhanu-sharma-VirtualBox\$ ls -l total 16 drwxr-xr-x 14 bhanu-sharma bhasharma 4096 Apr 19 17:21 Desktop -rw-r--r-- 1 bhanu-sharma bhasharma 10 Apr 19 16:51 README.md -rw-r--r-- 1 bhanu-sharma bhasharma 573 Apr 19 16:51 tool_install.sh -rw-r--r-- 1 bhanu-sharma bhasharma 1377 Apr 19 16:51 tool_install.sh -rw-r--r-- 1 bhanu-sharma bhasharma 14K Apr 19 16:51 tool_install.sh bhanu-sharma@bhanu-sharma-VirtualBox\$ ls -lh total 16K</pre>	<pre>bhanu-sharma@bhanu-sharma-VirtualBox\$ mkdir vlsi_project bhanu-sharma@bhanu-sharma-VirtualBox\$ ls mkdir vlsi_project vlsi_project bhanu-sharma@bhanu-sharma-VirtualBox\$</pre>				
CHANGING DIRECTORIES (cd deep, home)	FILE CONTENT VIEWING (cat)	OTHER BASIC COMMANDS				
<pre>bhanu-sharma@bhanu-sharma-VirtualBox\$ cd vlsi_project bhanu-sharma@bhanu-sharma-VirtualBox\$ cd bhanu-sharma@bhanu-sharma-VirtualBox\$</pre>	<pre>bhanu-sharma@bhanu-sharma-VirtualBox\$ cd Ubuntu bhanu-sharma@bhanu-sharma-VirtualBox\$ cat Ubuntu.txt Hello Ubuntu ! bhanu-sharma@bhanu-sharma-VirtualBox\$</pre>	<table border="0"><tr><td><code>>_</code> rm delete</td><td><code>>_</code> mv description</td><td><code>>_</code> cp commitment</td><td><code>>_</code> touch touch</td></tr></table>	<code>>_</code> rm delete	<code>>_</code> mv description	<code>>_</code> cp commitment	<code>>_</code> touch touch
<code>>_</code> rm delete	<code>>_</code> mv description	<code>>_</code> cp commitment	<code>>_</code> touch touch			



Basic Linux Commands Practiced

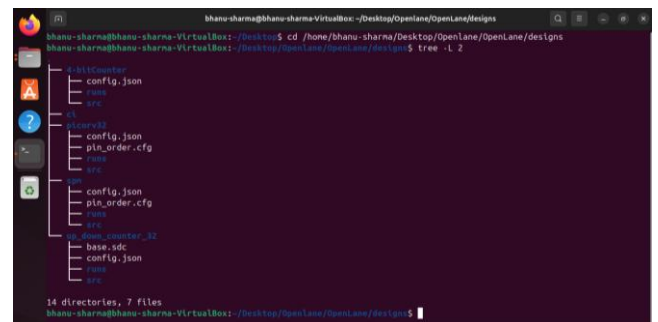
The following Linux commands were regularly used during the course:

Command	Purpose
pwd	Display the current working directory
Ls	List files and directories
cd	Change the current directory
mkdir	Create a new directory
rmdir	Remove an empty directory
cp	Copy files or directories
mv	Move or rename files
rm	Remove files or directories
touch	Create a new empty file
cat	Display file contents
less	View large text files
head	Display the beginning of a file
tail	Display the end of a file
grep	Search for specific text within files
find	Locate files and directories
chmod	Modify file permissions
echo	Display text or write data to files
history	Display previously executed commands
clear	Clear the terminal screen



Ubuntu 24.04 desktop running inside Oracle VirtualBox.

OpenLane project directory :



4.4 Git and GitHub

Git and GitHub were introduced as part of the project development and submission workflow. Git is a distributed version control system used to manage changes in source-code and project files, while GitHub provides a remote repository for storing and sharing project resources.

A GitHub repository was created for maintaining the project files and documentation. The repository is intended to contain the project report, Markdown documentation, assignments, Verilog source files, Vivado project files, and the ASIC physical design files and reports associated with the PicoRV32 implementation.



GitHub also provides a convenient platform for organizing the project deliverables and making the work accessible for evaluation and future reference.

The GitHub repository will serve as the central location for the project deliverables required for the SURE Trust submission.

GitHub Repository:

3.2 Design and Technology Setup

The PicoRV32 RISC-V processor was selected as the design under study for the physical design implementation. PicoRV32 is a compact 32-bit RISC-V processor core described using synthesizable Verilog RTL. The RTL design was implemented using an open-source digital physical design flow based on the SKY130 process design kit (PDK).

The physical design flow was performed using OpenLane, with OpenROAD used for major physical implementation and analysis tasks. The SKY130 PDK provides the technology information and standard-cell libraries required for synthesis and physical implementation. The design flow utilizes technology files such as standard-cell LEF, Liberty timing libraries, and routing-related technology information to perform the different stages of physical design.

The complete EDA environment was configured on Ubuntu 24.04 LTS running inside Oracle VirtualBox. The project environment consisted of 4 GB RAM and 4 processor cores allocated to the virtual machine. The configured environment was used to execute the RTL synthesis, floorplanning, power planning, placement,

clock tree synthesis, routing, and timing-analysis stages.

The major design and technology setup used in the project is summarized below.

3.3 Design Inputs and Constraints

The physical design implementation was performed using the PicoRV32 RTL design and the SKY130A process design kit. The main design input was the synthesizable Verilog RTL file picorv32.v. The SKY130A PDK and the sky130_fd_sc_hd standard-cell library were used to provide the technology and standard-cell information required during synthesis and physical implementation.

Parameter	Specification
Processor Design	PicoRV32 RISC-V Core
RTL Language	Verilog
Technology	SKY130
Process Node	130 nm
Physical Design Flow	OpenLane
Physical Design Engine	OpenROAD
Operating System	Ubuntu 24.04 LTS
Virtualization Platform	Oracle VirtualBox
Allocated Memory	4 GB RAM
Allocated CPU	4 Processor Cores
Target Clock Frequency	100 MHz
Target Clock Period	10 ns



Parameter	Configuration
Design	PicoRV32
RTL File	picorv32.v
PDK	sky130A
Standard-Cell Library	sky130_fd_sc_hd
Clock Port	Clk
Clock Period	10 ns
Target Frequency	100 MHz
Core Utilization	60%
Floorplan Aspect Ratio	0.7
Core PDN Ring	Enabled
IO_PCT	0.5
Horizontal I/O Metal	Metal 4
Vertical I/O Metal	Metal 4
Pin Order Configuration	pin_order.cfg

The design was configured with the clock port clk and a clock period of 10 ns, corresponding to a target frequency of 100 MHz. The initial floorplan was configured with a core utilization of 60% and an aspect ratio of 0.7. A core power ring was enabled as part of the power distribution network configuration.

The I/O configuration was controlled using the IO_PCT parameter and a dedicated pin-order configuration file. For the implementation studied in this configuration, IO_PCT was set to 0.5. Metal 4 was selected for both horizontal and vertical I/O pin placement. These

physical-design parameters were configured to establish the initial floorplan and I/O arrangement before proceeding with placement, clock tree synthesis, routing, and timing analysis.

The major design inputs and physical-design constraints are summarized below.

3.4.1 RTL Synthesis

The PicoRV32 processor RTL was used as the starting point of the physical design flow. The RTL source file picorv32.v was synthesized using Yosys within the OpenLane environment, with picorv32 configured as the top-level design. The synthesis flow processed the RTL hierarchy and retained the logic required by the selected PicoRV32 configuration while removing unused modules and redundant logic during the optimization stages.

During synthesis, Yosys performed RTL process conversion and optimization, including conversion of conditional logic into multiplexing structures, creation of sequential elements, removal of dead logic, and pruning of redundant assignments. The synthesis checks also verified that no unintended latches were inferred. The resulting logical design was then technology-mapped to the **SkyWater SKY130A sky130_fd_sc_hd standard-cell library**.

The synthesis stage generated the technology-mapped netlist and associated timing information required for the subsequent physical implementation stages. The synthesized netlist was then used as the input for floorplanning, where the physical dimensions, I/O arrangement, and power-distribution structure of the PicoRV32 design were established.

The detailed synthesis metrics, including mapped cell count, flip-flop count,



synthesized area, and connectivity checks, are presented separately in the Results and Discussion chapter.

3.4.2 Floorplanning

After synthesis, the technology-mapped PicoRV32 netlist was taken as the input for floorplanning. The floorplan was configured in OpenLane using a target core utilization of **60%** and an aspect ratio of **0.7**. These parameters were selected to define the initial core dimensions and provide sufficient placement area for the synthesized standard cells.

The power distribution structure was also enabled during floorplan generation by configuring the **core power ring**. The I/O pins were assigned according to the project-specific `pin_order.cfg` file. Metal 4 was selected for both horizontal and vertical I/O pin placement through the floorplanning configuration. The I/O placement configuration was checked during implementation to ensure that the ports were correctly positioned around the core before proceeding to placement.

The floorplanning stage therefore established the physical boundary of the PicoRV32 core, its I/O arrangement, and the initial power-distribution structure. The resulting floorplan was then passed to the placement stage for optimization and legalization of the standard cells.

Key floorplanning configuration:

Parameter	Value
Core Utilization	60%
Aspect Ratio	0.7
Core Power Ring	Enabled
I/O Horizontal Metal	Metal 4
I/O Vertical Metal	Metal 4
Pin Order File	pin_order.cfg
I/O Placement Parameter	IO_PCT

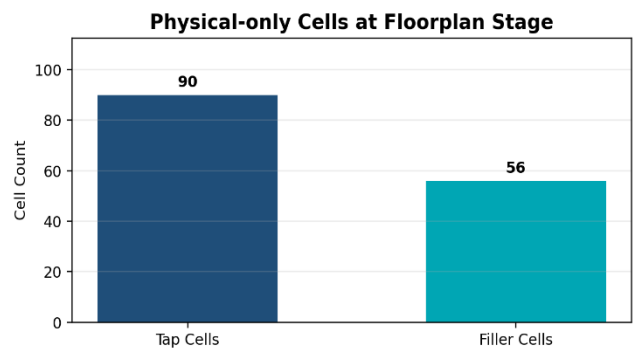


Figure: Tap and filler cell count at the floorplan stage.

Interpretation: Tap cells are the majority of physical-only cells and support well/substrate biasing; filler cells complete layout continuity and power rail connectivity.

The `pin_order.cfg` file is a straightforward configuration script used during the floorplanning stage (specifically by a tool called `io_placer`) to determine exactly which chip edge your input/output pins will land on, and in what order.



Here is how you format and use this file to organize your design's pins.

Structure of pin_order.cfg

The file is divided into four main direction blocks: #N, #S, #E, and #W. Under each block, you simply list the exact signal names from your Verilog top-level module, one per line. The tool reads them from top to bottom (usually left-to-right for North/South, and bottom-to-top for East/West).

Here is a typical layout template for a microcontroller core like the PicoRV32:

Code snippet

#N

clk

resetsn

trap

#S

mem_valid

mem_instr

mem_ready

#E

mem_wdata.*

mem_wstrb.*

#W

mem_addr.*

mem_rdata.*

- **#N (North):** Global controls (Clock, Reset), incoming memory read data, and Pico Co-Processor Interface (PCPI) response/IRQ inputs.
- **#S (South):** Processor status (trap) and standard memory transaction

outputs (mem_valid, mem_addr, mem_wdata, mem_wstrb).

- **#W (West):** Pico Co-Processor Interface (PCPI) outbound execution signals and End-of-Interrupt (eoi) outputs.
- **#E (East):** Look-Ahead memory interface outputs (mem_la_*) and hardware execution trace outputs (trace_*).

Here is the consolidated breakdown of your pin mapping configuration:

North Side (#N)

- clk / resetsn (System Clock & Active-Low Reset)
- mem_ready (Memory acknowledgement)
- mem_rdata[31:0] (32-bit Input Data Bus)
- pcpi_wr, pcpi_rd[31:0], pcpi_wait, pcpi_ready (Co-processor return interface)
- irq[31:0] (Interrupt request lines)

South Side (#S)

- trap (CPU Halt/Fault Indicator)
- mem_valid / mem_instr (Memory request and Instruction/Data flag)
- mem_addr[31:0] (32-bit Memory Address Bus)
- mem_wdata[31:0] (32-bit Output Data Bus)
- mem_wstrb[3:0] (4-bit Byte Write Strokes)

West Side (#W)

- pcpi_valid (Co-processor command valid flag)



- pcpi_insn[31:0] (Instruction routed to the co-processor)
- pcpi_rs1[31:0] / pcpi_rs2[31:0] (Source Register 1 & 2 values)
- eoi[31:0] (End-of-Interrupt outputs)

East Side (#E)

- mem_la_read / mem_la_write (Look-Ahead early cycle flags)
- mem_la_addr[31:0] (Look-Ahead address bus)
- mem_la_wdata[31:0] (Look-Ahead write data bus)
- mem_la_wstrb[3:0] (Look-Ahead write byte strobes)
- trace_valid / trace_data[35:0] (Hardware tracing/debugging bus)

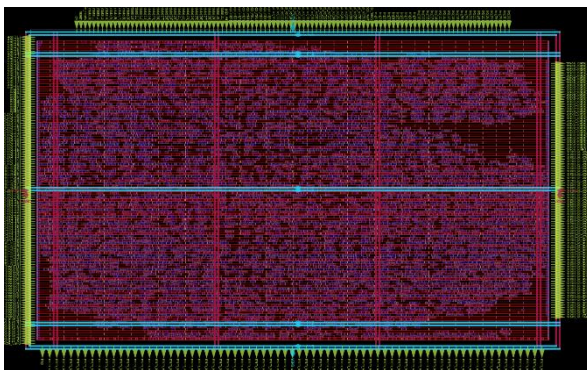


Figure : Floorplan and Placement

3.4.3 Power Distribution Network

After establishing the floorplan and I/O arrangement, the Power Distribution Network (PDN) was generated to provide the required power and ground connections to the standard cells throughout the PicoRV32 core. The PDN stage is responsible for creating a low-resistance power-delivery structure that connects the external power sources to the internal standard-cell rows.

A core power ring was enabled as part of the floorplan configuration. The PDN generation stage subsequently created the required power and ground structures across the core region and provided connections to the standard-cell power rails. The generated power network was integrated with the physical floorplan before placement and subsequent routing stages.

The PDN implementation was checked as part of the physical-design flow to ensure that the power network was properly generated and connected to the design. The completed PDN structure was then used during placement and routing, where the standard cells were physically connected to the power distribution network.

The quality of the generated power network was later evaluated using IR-drop analysis. The detailed IR-drop measurements and worst-case voltage degradation obtained from the implemented design are presented in the Results and Discussion chapter.

Physical-only cells commonly seen in Sky130

- **Tap cells**
(sky130_fd_sc_hd__tapvprvgn*_*)
 - Connect substrate and well regions to power/ground.
- **Endcap cells**
(sky130_fd_sc_hd__decap_*, endcap variants)
 - Placed at row boundaries.
- **Decap cells**
 - Provide local decoupling capacitance.
- **Filler cells**
(sky130_fd_sc_hd__fill_*)



- Fill gaps between standard cells.
- **Tie cells**
(sky130_fd_sc_hd__conb_*)
 - Generate constant logic 0 or logic 1.
- **Diode cells**
(sky130_fd_sc_hd__diode_*)
 - Used for antenna violation fixes.

3.4.5 Placement

After floorplanning and PDN generation, the synthesized standard cells of the PicoRV32 design were placed within the defined core region. The placement stage was performed using the OpenLane physical-design flow, with OpenROAD carrying out global placement, placement optimization, and detailed placement.

During global placement, the standard cells were initially distributed within the available core area while considering connectivity and timing requirements. Placement optimization was then performed to improve the physical arrangement of the cells and reduce the impact of interconnect on design performance. Detailed placement subsequently legalized the cell locations and ensured that the cells were positioned according to the available standard-cell rows and placement rules.

The placement stage was also used to evaluate the effect of the selected I/O arrangement on the physical implementation. Since the project investigated different IO_PCT configurations, the resulting placement and timing characteristics were compared for the selected configurations. This allowed the influence of I/O placement on critical

paths to be studied without changing the target clock requirement.

After successful placement, the legalized design was passed to the Clock Tree Synthesis (CTS) stage. The placement results and associated timing metrics were analyzed later to evaluate the quality of the physical implementation.

Tcl Script for finding the inverters in placement stage :

```
# Find all inverter cells matching the patterns
```

```
set inv_cells [get_cells -hierarchical -  
filter {ref_name =~ *inv*}]
```

```
# Initialize an array to keep track of counts
```

```
array set inv_counts {}
```

```
# Iterate through each inverter cell,  
print it, and count it
```

```
foreach i $inv_cells {
```

```
set ref [get_property $i ref_name]
```

```
puts "[get_property $i name] ->  
$ref"
```

```
if {[info exists inv_counts($ref)]} {
```

```
incr inv_counts($ref)
```

```
} else {
```

```
set inv_counts($ref) 1
```

```
}
```

```
}
```

Tcl Script for finding the buffers in placement stage :

```
# Find all buffer cells matching the patterns
```



```
set buf_cells [get_cells -hierarchical -
filter {ref_name =~ *buf*}]
```

```
# Initialize an array to keep track of
counts
```

```
array set buf_counts {}
```

```
# Iterate through each buffer cell, print
it, and count it
```

```
foreach b $buf_cells {
```

```
set ref [get_property $b ref_name]
```

```
puts "[get_property $b name] ->
$ref"
```

```
if {[info exists buf_counts($ref)]} {
```

```
incr buf_counts($ref)
```

```
} else {
```

```
set buf_counts($ref) 1
```

```
}
```

```
}
```

Buffer Type	Count
sky130_fd_sc_hd__buf_1	40
sky130_fd_sc_hd__buf_2	318
sky130_fd_sc_hd__buf_4	231
sky130_fd_sc_hd__buf_6	14
sky130_fd_sc_hd__buf_8	8
sky130_fd_sc_hd__clkbuf_1	1337
sky130_fd_sc_hd__clkbuf_2	26
sky130_fd_sc_hd__clkbuf_4	281
sky130_fd_sc_hd__clkbuf_8	58
sky130_fd_sc_hd__clkbuf_16	0

Inverter Type	Count
sky130_fd_sc_hd__clkinv_4	1
sky130_fd_sc_hd__inv_2	79
sky130_fd_sc_hd__inv_4	1
sky130_fd_sc_hd__inv_6	0
sky130_fd_sc_hd__inv_8	0

Category	Count
Regular Buffers	611
Clock Buffers	1702
Total Buffers	2313
Total Inverters	81
Total Buffer + Inverter Cells	2394

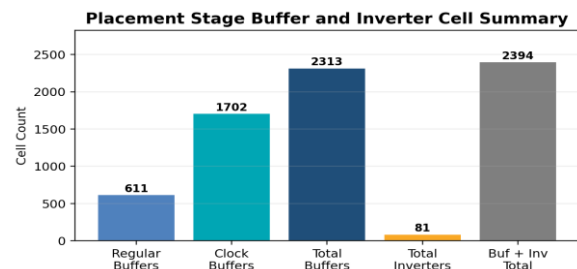


Figure: Placement stage buffer and inverter summary.

Interpretation: Clock buffers dominate the buffer/inverter population, which is expected for a clock-distributed processor implementation with many sequential elements.



Flip-Flop Drive Strength Distribution

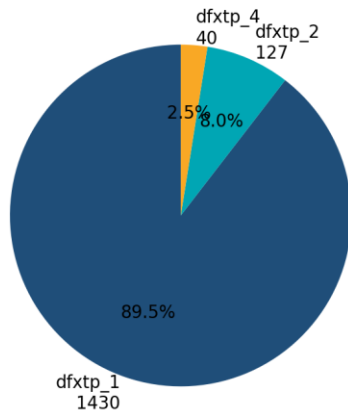


Figure: Flip-flop drive-strength distribution.

Interpretation: The flip-flop population is strongly dominated by dfxtp_1 cells, indicating that most sequential elements use the smallest drive strength available in the reported placement result.

Row Width	488520 DBU (488.52 μm)	1062 × 460 DBU
Row Orientation	Alternating N and FS	As shown in the DEF

NOR Gates	nor2_*, nor3_*, nor4_*, nor3b_*, nor4b_*	233
XOR/XNOR Gates	xor2_*, xnor2_*	119
AOI/OAI & Complex Gates	a21*, a22*, a31*, a32*, a211*, a221*, a311*, o21*, o22*, o31*, o211*, o221*, etc.	≈1480
Constant Cells	conb_1	106

Decoupling Capacitors	decap_3	250
Delay Cells	dlymetal6s2s_1	17
Tap Cells	tapvpwrvgnd_1	2349

Total Standard Cells = 12,254

3.4.6 Clock Tree Synthesis (CTS)

After successful placement, Clock Tree Synthesis (CTS) was performed to distribute the clock signal from the clock input to the sequential elements of the PicoRV32 design. The CTS stage was carried out using OpenROAD within the OpenLane physical-design flow.

The primary objective of CTS was to construct a balanced clock distribution network while controlling clock skew and minimizing the impact of clock insertion delay on timing. Clock buffers and inverters were inserted as required to drive the clock network across the placed standard cells. The generated clock tree was then propagated through the design to provide a physically realistic clock network for subsequent timing analysis.

The CTS configuration was implemented according to the project timing requirements, including a target clock skew of 5% of the 10 ns clock period. Therefore, the target skew used for CTS was 0.5 ns. Clock-tree optimization was performed after tree construction to improve the distribution of the clock signal and satisfy the physical and timing requirements of the design.

Following CTS, static timing analysis was performed to evaluate the setup and hold characteristics of the clocked paths with the generated clock network. The CTS results, including clock-tree characteristics, buffer insertion, skew, and



post-CTS timing, were analyzed as part of the overall physical-design evaluation.

The detailed CTS results and timing observations are presented in the Results and Discussion chapter.

Interpretation: CTS mainly adds high-drive clock buffers, especially clkbuf_16, to build a balanced and stronger clock distribution network.

Buffer Comparison (CTS → Routing)

Buffer Type	After CTS	After Routing	Change
buf_1	40	40	0
buf_2	318	109	-209
buf_4	231	284	+53
buf_6	14	37	+23
buf_8	8	17	+9
clkbuf_1	1337	1333	-4
clkbuf_2	26	26	0
clkbuf_4	281	387	+106
clkbuf_8	74	102	+28
clkbuf_16	119	121	+2

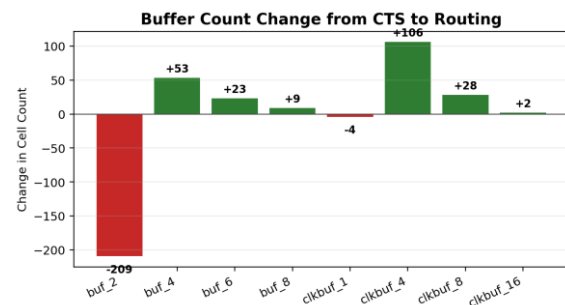


Figure: Buffer count change from CTS to routing.

Interpretation: Post-route optimization reduces many lower drive-strength buf_2 cells and increases stronger buffer variants to compensate for routed interconnect parasitics.

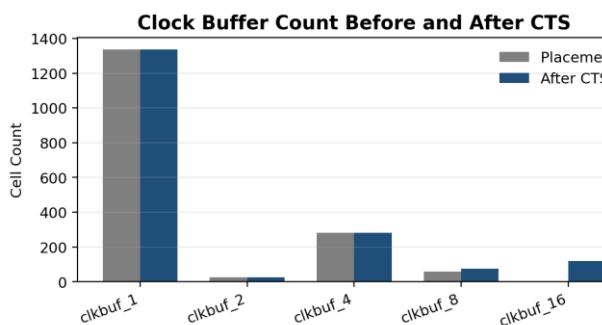
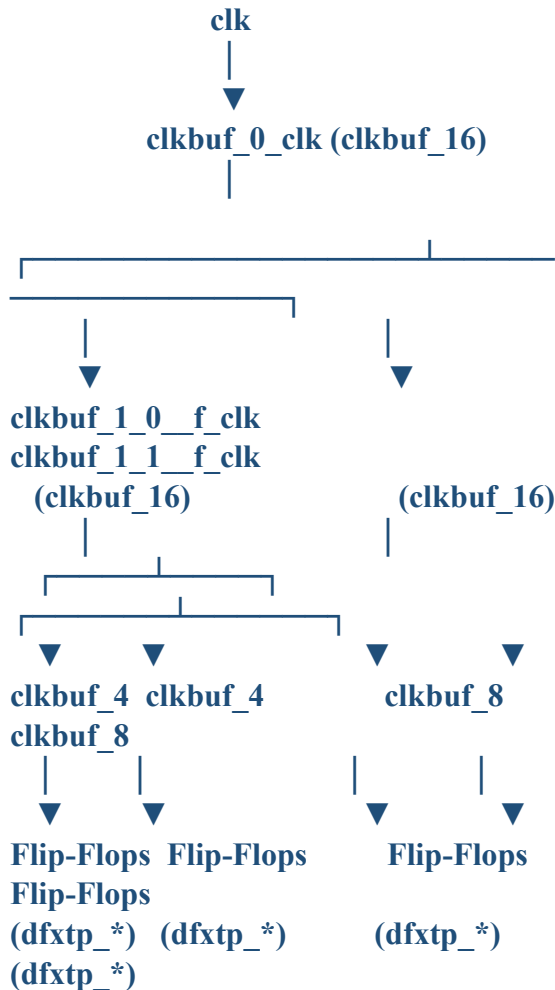


Figure: Clock buffer count before and after CTS.



Inverter Comparison (CTS → Routing)

Inverter Type	After CTS	After Routing	Change
clkinv_4	1	4	+3
inv_2	79	74	-5
inv_4	1	2	+1
inv_6	0	1	+1

3.4.7 Routing

After completion of Clock Tree Synthesis, the placed and clocked PicoRV32 design was taken through the routing stage. Routing was performed using OpenROAD within the OpenLane physical-design flow to establish physical interconnections between the synthesized standard cells, I/O pins, and the clock distribution network.

The routing process consisted of global routing followed by detailed routing. During global routing, routing resources were assigned to the different signal connections while considering the available metal layers and routing capacity. Detailed routing subsequently converted these routing decisions into physically valid connections while satisfying the technology-specific design rules.

The routing stage also included optimization of the routed design to address physical and timing requirements. The completed routing connected the signal, clock, and power-related structures across the implemented core and produced the routed physical database required for post-route analysis.

The routed design was subsequently checked for routing completion and physical connectivity. Post-routing timing and physical characteristics were then evaluated to determine the effect of the actual interconnect on design performance. Parameters such as routed wirelength,

setup and hold timing, and power-integrity characteristics were considered during the final evaluation.

The detailed routing results and post-route measurements are presented in the Results and Discussion chapter.

3.4.8 Static Timing and Physical Analysis

After routing, the implemented PicoRV32 design was subjected to post-route static timing and physical analysis to evaluate whether the generated layout satisfied the specified design requirements. Static Timing Analysis (STA) was performed using the clock constraint of 10 ns, corresponding to the target operating frequency of 100 MHz.

Both setup and hold timing characteristics were analyzed for the implemented design. Setup analysis was used to determine whether data could propagate through the combinational logic and reach the receiving sequential element within the required clock period. Hold analysis was used to verify that data remained stable for the required duration after the active clock edge. The worst negative slack (WNS) and total negative slack (TNS) were considered as the primary indicators for identifying timing violations.

The physical implementation was also evaluated using post-route parameters such as cell utilization, routed wirelength, clock-tree characteristics, and power-integrity information. IR-drop analysis was performed to examine the voltage degradation in the power distribution network and to identify the worst-case voltage drop within the implemented core.



The timing and physical analysis results obtained from the different stages of the implementation flow were compared to identify the major factors affecting the design performance. In particular, the effect of different I/O placement configurations was investigated while maintaining the same target clock period. This analysis helped evaluate the relationship between physical implementation choices and timing performance.

The detailed timing, wirelength, utilization, and IR-drop results obtained from the implemented design are presented and discussed in Chapter 4.

4.1 Synthesis Results

The PicoRV32 RTL was synthesized using Yosys within the OpenLane flow and technology-mapped to the SKY130A sky130_fd_sc_hd standard-cell library. The synthesis stage converted the RTL description into a gate-level representation suitable for subsequent physical implementation.

During synthesis, the RTL hierarchy was analyzed and optimization operations were performed to remove unused and redundant logic. The synthesis checks were completed successfully, with the final clean check reporting no remaining design problems. Sequential elements were inferred for the required processor registers and control logic, while combinational logic was mapped to the available standard cells.

The synthesized design resulted in a total synthesized cell area of approximately **100,370 μm^2** . The synthesis flow produced the technology-mapped netlist and associated timing information, which were subsequently used as inputs to the floorplanning and physical implementation stages.

The major synthesis results are summarized below.

Parameter	Result
Design	PicoRV32
Synthesis Tool	Yosys
Standard-Cell Library	sky130_fd_sc_hd
Synthesized Cell Area	$\approx 100,370 \mu\text{m}^2$
Final Synthesis Check Problems	0
Target Clock Period	10 ns
Target Frequency	100 MHz

The synthesis results confirm that the PicoRV32 RTL was successfully converted into a technology-mapped gate-level design and was suitable for further physical implementation. Detailed timing and physical metrics obtained after placement, CTS, and routing are discussed in the subsequent sections.

4.2 Floorplanning and Utilization Results

The floorplanning stage was performed with a target core utilization of 60% and an aspect ratio of 0.7. The objective was to provide sufficient placement area for the synthesized PicoRV32 standard cells while maintaining a compact and routable core region.

The floorplan generated by OpenLane established the core boundary, standard-cell placement region, I/O locations, and power-distribution structure. The core power ring was also incorporated as specified in the physical-design configuration.



The utilization observed during the physical implementation was monitored after floorplanning and placement. The final placement results showed a utilization of approximately **62%**, indicating that the available core area was being effectively occupied while retaining routing space for interconnects.

Parameter	Result
Target Core Utilization	60%
Floorplan Aspect Ratio	0.7
Observed Placement Utilization	≈ 62%
Core Power Ring	Enabled
I/O Configuration	IO_PCT = 0.5
I/O Metal	Metal 4

The small difference between the target utilization and the observed placement utilization is associated with the physical implementation of the synthesized standard cells and the resulting placement arrangement. The floorplan provided the physical foundation for subsequent placement optimization, CTS, and routing.

Figure 4.1 should be the **floorplan screenshot** from your final run. A suitable caption is:

Figure 4.1: Floorplan of the PicoRV32 design showing the core region, I/O placement, and power-distribution structure.

The floorplan and utilization results indicate that the selected floorplanning parameters produced a successfully placeable physical design, which was

subsequently taken through CTS and routing.

4.3 Power Distribution Network and IR-Drop Results

The Power Distribution Network (PDN) was generated as part of the physical implementation with the core power ring enabled. The purpose of the PDN was to provide a reliable power and ground distribution path to the standard-cell rows throughout the PicoRV32 core.

Post-implementation IR-drop analysis was performed to evaluate the voltage degradation caused by resistance in the power distribution network. The analysis showed a worst-case received VDD of approximately **1.79947 V** at cell **_229_**. The corresponding IR drop was approximately **0.000530 V**, resulting in a voltage degradation of approximately **0.0294%**.

Parameter	Result
Core Power Ring	Enabled
Worst-Case Cell	<u>_229_</u>
Received VDD	1.79947 V
IR Drop	0.000530 V
Voltage Degradation	0.0294%

The measured voltage degradation is very small compared with the nominal supply voltage, indicating that the implemented power distribution network provides adequate voltage delivery to the analyzed standard-cell locations. No significant power-integrity issue was indicated by the observed worst-case IR-drop result.



Figure 4.2 should show the **IR-drop analysis result or OpenROAD power-integrity view** from the final run.

Figure 4.2: IR-drop analysis of the implemented PicoRV32 design showing the worst-case voltage degradation.

The IR-drop analysis therefore confirms that the generated PDN maintained a stable supply voltage across the analyzed portion of the implemented design. Further timing and routing results are discussed in the following sections.

4.4 Placement Results

Following floorplanning and PDN generation, the synthesized standard cells of the PicoRV32 design were placed within the defined core region.

OpenROAD performed global placement, placement optimization, and detailed placement to obtain a legalized physical arrangement of the standard cells.

The placement stage resulted in a physically valid distribution of the synthesized logic within the available core area. Placement optimization considered cell connectivity and timing requirements to improve the physical arrangement before the design was passed to Clock Tree Synthesis (CTS).

The placement results also demonstrated the influence of the I/O configuration on the physical implementation. In particular, the project investigated different IO_PCT values while keeping the clock period fixed at **10 ns**. The comparison showed that the physical arrangement associated with the I/O configuration had a noticeable effect on timing performance.

For the investigated IO_PCT configurations, the observed placement-stage setup slack was:

I/O Configuration	Worst Setup Slack	Worst Hold Slack
IO_PCT = 0.2	+2.98 ns	+0.17 ns
IO_PCT = 0.5	-1.43 ns	+0.18 ns

The results show that the setup margin decreased significantly when the I/O configuration was changed from IO_PCT = 0.2 to IO_PCT = 0.5, while the hold margin remained positive and changed only slightly. This indicates that the physical arrangement of the design can have a substantial influence on critical setup paths even when the target clock period remains unchanged.

Figure 4.3 should show the **placed PicoRV32 design in OpenROAD**, preferably the final placement view.

Figure 4.3: Placed PicoRV32 design showing the distribution of standard cells within the core region.

The legalized placement was subsequently used as the input for Clock Tree Synthesis. The effect of CTS and the resulting clock-tree characteristics are discussed in the next section.

4.5 Clock Tree Synthesis Results

Clock Tree Synthesis (CTS) was performed after placement to construct the clock distribution network for the PicoRV32 design. The objective was to distribute the clock signal to the sequential elements while maintaining the required clock skew and providing sufficient drive strength across the design.



The CTS stage inserted a large number of clock buffers and associated buffering cells to distribute the clock signal across the placed design. The observed clock-buffer distribution included different drive-strength cells, indicating that the clock network was optimized according to the load and physical distribution of the sequential elements.

For the implemented configuration, the major clock-buffer counts observed after CTS were:

Cell Type	Count
clkbuf_1	1339
clkbuf_2	29
clkbuf_4	249
clkbuf_8	75

The CTS results demonstrate that significant clock buffering was required to distribute the clock across the PicoRV32 design. Higher-drive clock buffers were used where required to handle larger loads and maintain the clock distribution quality.

The CTS timing analysis was performed using the project requirement of a **10 ns clock period** and a target clock skew of **0.5 ns**, corresponding to 5% of the clock period. The generated clock tree was subsequently passed to the post-CTS timing and routing stages for further optimization and analysis.

Figure 4.4 should show the **CTS clock tree or clock distribution view** from the final implementation.

Figure 4.4: Clock Tree Synthesis result showing the clock distribution network of the PicoRV32 design.

Overall, the CTS stage successfully generated the clock distribution network required for the placed design. The effect of the generated clock tree on setup and hold timing is considered in the subsequent timing analysis.

4.6 Routing and Wirelength Results

After completion of placement and Clock Tree Synthesis, the PicoRV32 design was taken through the routing stage. Global routing followed by detailed routing was performed using OpenROAD to establish the physical interconnections between the standard cells, I/O pins, and clock distribution network.

The routing process successfully generated the physical interconnects required by the implemented design. Post-route analysis was then used to evaluate the resulting interconnect characteristics. One of the primary physical parameters considered was the total routed wirelength, since excessive interconnect length can increase parasitic effects and consequently affect timing and power characteristics.

For the implemented design, the extracted total routed wirelength was approximately **6719 μm** .

Parameter	Result
Routing Flow	OpenLane / OpenROAD
Routing Stages	Global Routing + Detailed Routing
Total Routed Wirelength	$\approx 6719 \mu\text{m}$
Design	PicoRV32



Parameter	Result
Technology	SKY130A

The obtained wirelength represents the physical interconnect required to connect the placed and clocked standard-cell design. The routed layout was subsequently used for post-route static timing analysis and other physical checks. Since interconnect parasitics become available after routing, the post-route timing results provide a more realistic assessment of the implemented design compared with earlier placement-stage estimates.

Figure 4.5 should show the **final routed layout of the PicoRV32 design**.

Figure 4.5: Final routed layout of the PicoRV32 design showing the physical interconnections.

The post-route timing results, including setup and hold slack, are discussed in the following section.

4.7 Post-Route Static Timing Analysis

Post-route Static Timing Analysis (STA) was performed to evaluate the timing performance of the implemented PicoRV32 design after routing. The analysis was carried out using the specified clock period of **10 ns**, corresponding to a target operating frequency of **100 MHz**. Both setup and hold timing were evaluated using the worst slack values obtained from the implementation flow.

A comparison was performed for two I/O placement configurations, $IO_PCT = 0.2$ and $IO_PCT = 0.5$, while keeping the clock requirement unchanged. The results are summarized below.

Parameter	$IO_PCT = 0.2$	$IO_PCT = 0.5$
Clock Period	10 ns	10 ns
Target Frequency	100 MHz	100 MHz
Worst Setup Slack	+2.98 ns	-1.28 ns
Worst Hold Slack	+0.17 ns	+0.19 ns
WNS	0.00 ns	-1.28 ns
TNS	0.00 ns	-2.48 ns
Setup Status	Met	Violated
Hold Status	Met	Met

The comparison shows a significant difference in setup timing between the two I/O configurations. With $IO_PCT = 0.2$, the design maintained a positive setup margin of **2.98 ns**, with no reported total negative slack. When IO_PCT was changed to 0.5, the worst setup slack became **-1.28 ns**, resulting in a setup timing violation. In contrast, the hold slack remained positive for both configurations and therefore did not present a hold-time violation.

The results indicate that the physical I/O placement configuration had a significant influence on the critical setup paths of the design. Since the target clock period was kept constant at 10 ns for both cases, the observed timing difference is associated with changes in the physical implementation rather than a change in the required operating frequency.

The $IO_PCT = 0.5$ configuration therefore represents a more challenging timing condition for the implemented design,



whereas the IO_PCT = 0.2 configuration provided a larger setup timing margin.

Figure 4.6 should show the **post-route STA report or timing-path report** containing the worst setup slack.

Figure 4.6: Post-route static timing analysis showing the worst-case setup timing result.

The timing analysis demonstrates the importance of physical implementation choices in achieving timing closure. The final timing result is considered together with the routing and power-integrity results to evaluate the overall quality of the implemented design.

Learning and Reflection

Working on the PicoRV32 physical design project provided practical exposure to the complete RTL-to-GDSII implementation flow and helped connect theoretical concepts of VLSI physical design with an actual processor design. The project provided an opportunity to work with an open-source EDA environment and understand how a synthesized RTL design is progressively transformed into a physically implemented layout.

One of the major learnings from the project was understanding the relationship between different physical-design stages. Initially, synthesis appeared to be mainly a logical transformation of RTL into standard cells; however, the subsequent floorplanning, placement, CTS, and routing stages

demonstrated that physical decisions can directly affect timing and overall design quality. In particular, the experiments with different I/O placement configurations showed that changing a physical implementation parameter can significantly influence setup timing even when the target clock period remains unchanged.

The project also provided practical experience in interpreting EDA tool reports rather than relying only on whether a flow completed successfully. Timing reports, utilization, clock-tree information, routing wirelength, and IR-drop results were examined to understand the actual quality of the implementation. This helped develop the ability to distinguish between a successful tool run and a design that actually satisfies its required constraints.

Another important learning was the debugging of physical-design flow issues. Problems encountered during I/O placement and timing closure required examination of configuration files, tool logs, and intermediate results. This improved understanding of how configuration parameters, design constraints, and physical implementation are interconnected.

From a technical perspective, the project strengthened my understanding of synthesis, floorplanning, power planning, placement, CTS, routing, static timing analysis, and IR-drop analysis. It also improved my familiarity with OpenLane, OpenROAD, Yosys, SKY130A,



standard-cell libraries, and the interpretation of physical-design reports.

Overall, the project provided a practical understanding of the challenges involved in achieving timing closure and physical implementation. The experience has strengthened my interest in VLSI physical design and provided a foundation for further learning in areas such as timing optimization, congestion analysis, power optimization, parasitic-aware analysis, and advanced physical-design methodologies.

Conclusion and Future Scope

This project successfully demonstrated the implementation of the PicoRV32 32-bit RISC-V processor through an open-source RTL-to-GDSII physical design flow using OpenLane, OpenROAD, Yosys, and the SKY130A technology. The design was taken through the major physical-design stages, including synthesis, floorplanning, power distribution network generation, placement, clock tree synthesis, routing, and post-route analysis.

The project provided practical understanding of how logical design characteristics and physical implementation decisions influence the final performance of a digital IC. The implemented design achieved a routed wirelength of approximately **6719 μm** , while the IR-drop analysis showed a worst-case voltage degradation of only approximately **0.0294%**. The timing analysis demonstrated that the design

maintained positive hold timing, while the $\text{IO_PCT} = 0.5$ configuration resulted in a setup violation of approximately **-1.28 ns** at the target clock period of 10 ns.

The comparison of different I/O placement configurations highlighted the importance of physical-design optimization for timing closure. The results demonstrate that achieving a successful physical implementation is not limited to completing the RTL-to-GDSII flow; the resulting layout must also be evaluated against timing, routing, utilization, and power-integrity requirements.

Overall, the project provided hands-on experience with an industry-relevant physical-design methodology and strengthened the understanding of the relationship between synthesis, physical implementation, timing analysis, and design optimization.

Future Scope

The present implementation can be further improved by exploring additional physical-design optimization techniques. The following areas can be considered for future work:

1. **Timing Optimization:** Further optimization of critical paths can be performed to eliminate the observed setup timing violation and achieve timing closure at 100 MHz.
2. **I/O Placement Optimization:** Additional I/O placement configurations can be investigated to identify an arrangement that provides better timing without unnecessarily increasing the physical area.
3. **Placement and Routing Optimization:** Placement density, routing resources, and cell



locations can be optimized to reduce interconnect delay and improve overall routability.

4. **Clock-Tree Optimization:** CTS parameters and buffer distribution can be further optimized to reduce clock skew and clock insertion delay while controlling clock-tree area and power.
5. **Power Optimization:** Dynamic and leakage power can be analyzed in greater detail, followed by optimization techniques to reduce overall power consumption.
6. **Advanced Physical Analysis:** Further work can include detailed congestion analysis, parasitic-aware timing analysis, IR-drop analysis across multiple operating conditions, and design-rule verification.
7. **Technology Scaling:** The same physical-design methodology can be explored using more advanced technology nodes and different standard-cell libraries to study the impact of technology scaling on area, timing, power, and routing.

The project therefore provides a foundation for further investigation into timing closure, physical optimization, and complete ASIC implementation methodologies.